CaseAssessment: Never Lose a Heartbeat

Format: individual. **Duration:** __ hours. **Date:** __. **Contact:** __

1. The problem

A small bedside device records two sensors continuously, sometimes for hours. It works in hospitals where WiFi drops without warning, power can be cut, and nobody watches the device.

The task is to build a data pipeline, on a Raspberry Pi-class Linux board and a free cloud service, that:

- **never loses data:** every sample is saved safely on the device;
- **processes data live when the network is available:** the data streams to the cloud and is processed there within seconds (**online processing**);
- **keeps working when the network is not available:** the device processes the data itself (**offline processing**), then sends everything to the cloud when the network returns.

A provided mock service imitates the sensors (section 6).

Where it runs. The solution must show that it works on a small device and a real cloud service:

- **Device side:** an ARM64 Raspberry Pi environment.
    - This can be a physical Raspberry Pi, an online Raspberry Pi emulator, or an ARM64 virtual machine (for example, a free-tier ARM64 cloud instance or a QEMU Raspberry Pi OS image).
    - It must be limited to Raspberry Pi-class resources: at most 4 CPU cores and 2 GB of memory.
    - The mock service and the pipeline both run there.
- **Cloud side:** any free cloud provider or free tier, for receiving, storing and processing the data.

Judging runs on a real Raspberry Pi 4 and the participant's cloud account.

2. What to build

1. **Collect and save.**
    - Read both sensors from the mock service continuously.
    - Save every sample on the device disk.
    - Saved data must survive program crashes, reboots and sudden power cuts.

2. **Online mode (network available).**
    - Stream the data to the cloud continuously.
    - Process it in the cloud (section 3), producing a result for each second within 5 seconds.

3. **Offline mode (no network).**
    - Keep collecting and saving.
    - Process the data on the device with the same logic (section 3).

4. **Switch automatically.**

   - Move to offline mode when the network is lost, and back to online mode when it returns, each within 10 seconds.
   - No sample may be lost or duplicated because of a switch.

5. **Catch up.** When the network returns, send every saved sample that the cloud does not yet have, so that the cloud ends up with a complete copy of the session, with every sample exactly once.

6. **Handle loss honestly.**

   - If any samples are lost, record exactly which ones (sensor, sample index range) and why.
   - Never fill a gap with invented values.

The design and tools are free to choose: any language, any free cloud service.

3. Processing logic (same online and offline)

Split the data into one-second windows: window `w` holds samples `400w` to `400w + 399` of each sensor.

For each window, produce:

Output	Rule
`missing_1`, `missing_2`	Number of samples of sensor 1 and sensor 2 not available in the window
`complete`	`true` if both are 0
`mean`, `min`, `max`	Per channel, over the available samples (`null` if none)
`magnitude`	Mean of $\sqrt{(ch1^2 + ch2^2)}$ over the available sensor 2 samples
`heart_rate`	Beats per minute from sensor 1, channel 2 (rule below)
`mode`	`online` (processed in the cloud) or `offline` (processed on the device)

Heart-rate rule.

1. **Peak:** sample `i` is a peak when all four of these hold:

   - the 800 samples before it, and the sample after it, are available;
   - it is larger than both neighbours;
   - it is larger than half of the maximum of the 800 samples before it;
   - it is at least 100 samples after the previous peak.

2. **Rate:** for window `w`, take the peaks in the last 5 seconds (samples `400(w − 4)` to `400(w + 1) − 1`). Then `heart_rate = 60 ÷ (mean gap between consecutive peaks, in seconds)`.

3. **No rate:** `heart_rate` is `null` if `w < 4`, if any sample in those 5 seconds is missing, or if there are fewer than 2 peaks.

Round decimal results to 1 decimal place.

4. Completion criteria

A submission is complete when all of the following are true. The judges test them automatically.

#	Criterion	How it is checked
1	Every sample the device received is on its disk, exactly once, unchanged	Device export compared with the mock's ground truth
2	Every sample that is not on disk is listed as lost, with a reason	Same comparison: no silent loss allowed
3	Data survives a crash, a reboot and a power cut. A power cut loses at most the stated amount (stated by the participant, in seconds)	Processes killed and power pulled during a run
4	In online mode, each second's result appears in the cloud within 5 seconds	Results timestamps
5	Mode switches within 10 seconds of the network changing, without loss or duplication	Network cut and restored during a run
6	After the network returns (outages of up to 2 hours), the cloud copy is complete within 15 minutes	Cloud export compared with ground truth
7	Results follow section 3 exactly	Compared with a reference implementation
8	Runs within small-device limits (ARM64, at most 4 cores and 2 GB of memory), with CPU, memory, disk and network use reported	Run on a Raspberry Pi 4; reported figures checked

Required commands (one program, `pipeline`, on the device):

- `pipeline status`: current mode, samples saved, samples in the cloud, and backlog. Printed as one JSON line.
- `pipeline export --session ID --from device` (or `--from cloud`): data as JSON lines, one per sample, `{"sensor": 1, "index": 120000, "values": [720, 1200, 1560]}`, then one line per loss, `{"sensor": 2, "from_index": 4000, "to_index": 4399, "reason": "..."}`.
- `pipeline results --session ID`: one JSON line per window and mode, with the section 3 outputs.

Scoring (100 points):

Area	Points
Data safety on the device (criteria 1–3)	35
Online mode and switching (criteria 4–5)	25
Complete cloud copy (criterion 6)	20
Correct results (criterion 7)	10
Design note, code quality and resource use (criterion 8)	10

Deductions: 5 points for each silent loss or unflagged wrong value the judges find, up to 30. Using a paid service disqualifies the submission.

5. What to submit

1. Code repository with a README to install and run on a fresh board.

2. A design note of at most 2 pages:

- the architecture;
- what is guaranteed and what is not;
- the maximum data loss on a power cut, and why.

3. Evidence of the failure tests run, and their results.

4. Resource figures over 1 hour in online mode, measured on the ARM64 environment used: average CPU, peak memory, megabytes written to disk and megabytes sent over the network. State which environment was used.

5. A demo video of at most 5 minutes, showing a network cut, a process kill and a power cut.

Rules:

- Free services and open-source software only.
- Do not modify the mock service.
- All data is synthetic.
- Disclose any AI tools used.

6. The mock service (provided)

`biosignal-mock` runs on the board and imitates the device's two sensors. Download: __ (ARM64 and x86-64).

```
biosignal-mock --mode signal --seed 7 --duration 3600 --port 7100
```

6.1 Data

	Sensor 1	**Sensor 2**
Channels	3	2
Rate	400 samples per second	400 samples per second
Values	Signed 24-bit integers	Signed 24-bit integers
Index	Sample number from 0	Sample number from 0

Both sensors start together, so sample `i` of sensor 1 and sample `i` of sensor 2 are taken at the same moment.

Two modes:

- `--mode signal`: realistic synthetic data.
  - Sensor 1, channel 2 looks like an ECG, with one sharp peak per heartbeat. The heart rate is 60 bpm for minutes 0–2, 90 for minutes 2–4, then 72 for minutes 4–6, repeating.
  - Sensor 1, channels 1 and 3 are channel 2 × 0.6 and × 1.3.
  - Sensor 2 has channel 1 near 400,000 and channel 2 near 150,000, varying slowly.
- `--mode ramp`: predictable values, for checking that nothing is lost or changed.
  - Sensor 1 channel `c` at sample `i` = `(3i + c) mod 8,388,608`.
  - Sensor 2 channel `c` = `(2i + c) mod 8,388,608`.

The same `--seed` always produces the same values.

6.2 Reading data

Newline-delimited JSON over TCP on the given port, one client at a time.

Request the next data for a sensor:

```
{"cmd": "read", "sensor": 1}
```

Reply (at most 1 second of data):

```
{"session_id": "5f0c2a9e", "sensor": 1, "first_index": 120000, "count": 3, "lost": [],
 "session_end": false, "crc32": "f9a5bf62", "data": [[720, 1200, 1560], [708, 1180, 1534], [726, 1210, 1573]]}
```

Field	Meaning
`session_id`	Changes when the mock restarts; indexes then start again from 0
`first_index`, `count`	Index of the first sample in `data`, and the number of samples
`lost`	Ranges the mock could not deliver, for example `{"from_index": 4000, "to_index": 4399, "reason": "buffer_overflow"}`
`session_end`	`true` in the last reply of a session
`crc32`	CRC-32 (zlib) of the `data` text exactly as sent, from `[` to the matching `]`. A reply with a wrong value is corrupt and must not be used
`data`	One array per sample: 3 values for sensor 1, 2 for sensor 2

Re-read data, for example after a corrupt reply:

```
{"cmd": "read", "sensor": 1, "from_index": 120000}
```

Buffer. The mock keeps only the last **10 seconds** of each sensor. Data not read in time is lost and reported in `lost`.

6.3 Faults the judges inject

During judging, the mock may:

- stop answering for a while;
- send a corrupt reply (wrong `crc32`);
- send the same reply twice;
- stop sending one sensor for a while (reported in `lost`);
- restart, which starts a new session. Unread data of the old session is then lost and must be listed, with `"to_index": null` when its end is unknown.

The judges also cut the network, kill processes, pull the power, fill the disk and change the system clock.

A test tool, `check_ground_truth`, is provided with the mock. It compares an export with the ground truth, so a pipeline can be tested before submission.